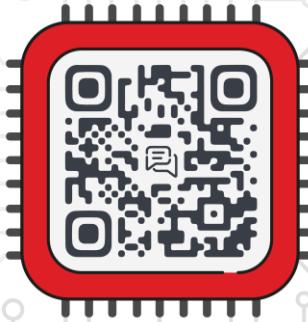




16



2



2400 sp

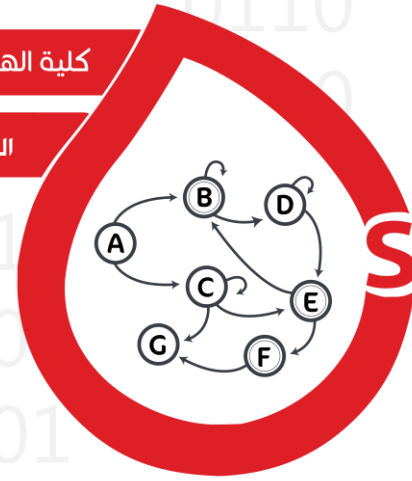
نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Blind Search

د. باسل الخطيب



RB Informatics; 12/10/2025

مقدمة:

درسنا في مقررات سابقة بعض خوارزميات البحث مثل DFS و BFS و الـ Dijkstra في مقرر خوارزميات 2 . في هذه المحاضرة سنقوم بمراجعة هذه الخوارزميات بالإضافة إلى خوارزميات جديدة و طرح بعض المفاهيم الهامة

1- عملية البحث (search process):

بعد أن نقوم بتمثيل المشكلة ← حالات (states)، هدف (goal)، عمليات (actions). يبدأ الـ AI في البحث بداخل الـ state space. المشكلة هنا تكمن ب أن كل عملية توسيع expansion تولد عدة حالات جديدة بحيث كل حالة تقوم بتوليد عدة حالات وهكذا مما يضاعف عدد الـ node بداخل الـ state space فيحدث ما يعرف ب انفجار فضاء البحث combinatorial explosion

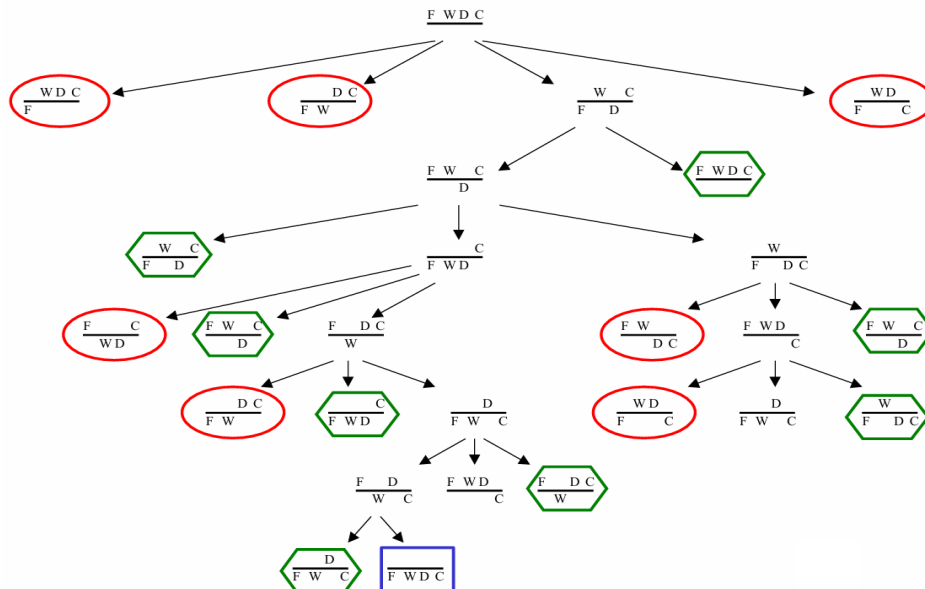
لنعد لمسألة الـ FWGC في المحاضرة السابقة

(F: Famer, W: Wolf, D: Dog, C: Cat : FWDC مسألة)

من الحالة الأولى = الكل في ضفة النهر اليسرى ونريد نقلهم للضفة اليمنى

يمكن تنفيذ 4 عمليات مختلفة كل عملية تولد حالة جديدة بعد مستويين فقط من التوسع يصبح لدينا عشرات الحالات المختلفة لذلك نحن نحتاج إلى طريقة منظمة Systematic search للسيطرة على

هذا التوسع



2- شجرة البحث (search tree):

عامل التفرع b $\Leftrightarrow$ Branching factor (effective)

هو متوسط عدد العقد الجديدة التي يتم إنشاؤها عند توسيع عقدة واحدة.

عمق المسار d $\Leftrightarrow$ Depth

طول المسار من العقدة الجذرية في شجرة البحث إلى العقدة الهدف. أي عدد الخطوات المطلوبة للوصول إلى الهدف.

عمق فضاء البحث m $\Leftrightarrow$ maximum

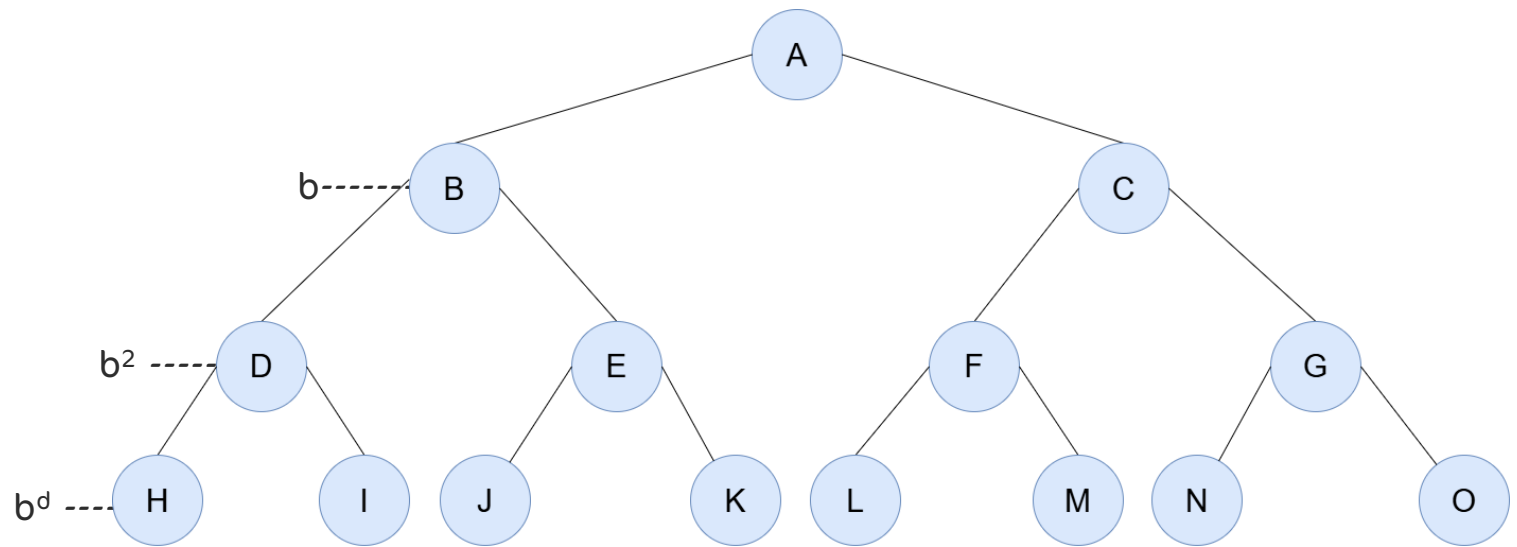
أقصى عمق ممكن في فضاء البحث كله.

كل مستوى في الشجرة يمثل خطوة جديدة من البحث (عمق أكبر).

عدد العقد الكلي حتى العمق d هو:

$$O(b^d) = 1 + b + \dots + b^d$$

عدد الحالات يزداد بشكل أسي مع العمق وهذه هي مشكلة الانفجار في البحث.



Fringe (Frontier):

سنشرح هذا المفهوم في
هذه المحاضرة

Branching factors example:

1. 8-puzzle

$$b = 2.13$$

عند العمق $d=20$ يكون لدينا عدد حالات حوالي

$$b^d = 2.13^{20} \text{ أي } 3.7 \text{ مليون حالة}$$

عدد الحالات الممكنة عند عمق وسطي هي 181,400

2. Rubik's cube

$$b = 13.34$$

عند عمق تقريبي $d=18$

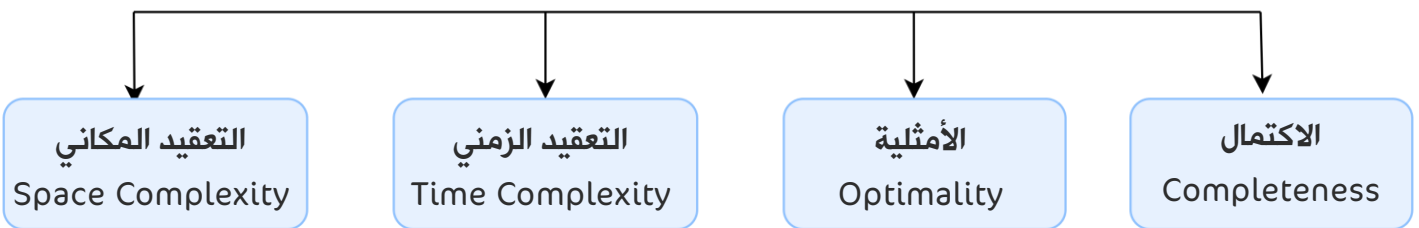
$$\text{عدد الحالات يكون حوالي } 9.01 \times 10^{17}$$

3. chess

$$b = 35$$

يوجد عدد حالات تقريباً 10^{120}

معايير تقييم خوارزميات البحث



- Completeness الاكتمال

هل تضمن الخوارزمية إيجاد الحل إن كان موجوداً. مثلاً خوارزمية BFS تجد الحل، لكن الـ DFS لا تضمن ذلك.

- Optimality الأمثلية

هل تضمن إيجاد أفضل حل.

- Time Complexity التعقيد الزمني

كم من الوقت تحتاج الخوارزمية لإيجاد الحل. أو كم عدد العقد التي تولدها الخوارزمية.

- Space Complexity التعقيد المكاني

كم الحجم الذي تحتاجه لإيجاد الحل. أو العدد الاعظمي للعقد التي تم تخزينها في الذاكرة.

Tree Search Approach:

منهج أو طريقة عمل شجرة البحث

الفكرة في عمل أي خوارزمية هي استكشاف فضاء الحالات بتوسيع العقد وإنشاء العقد الفرعية.
لنتعرف على الفرق بين الـ node والـ state:

1. الحالة (state):

تمثل الوضع الفعلي للمسألة أو اللعبة الـ physical configuration
مثال: في مسألة الـ 8-Puzzle الحالة هي ترتيب البلاطات داخل الشبكة في كل مرة.

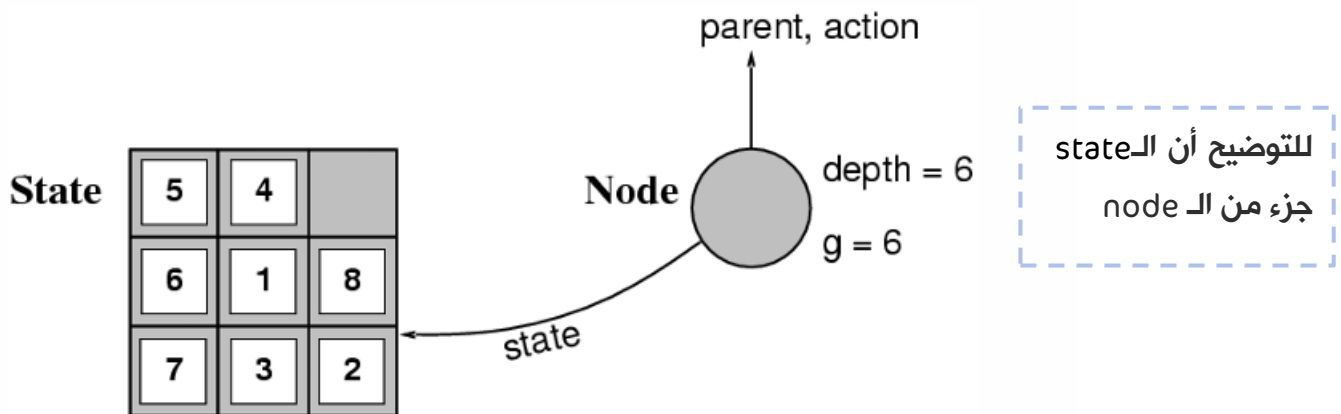
2. العقدة (node):

هي هيكل بيانات Data Structure لا تمثل الوضع الحالي فقط بل أيضاً معلومات إضافية نحتاجها في خوارزميات البحث.

❖ **State** هي الوصف المجرد للمكان أو للعالم الذي نحن فيه لا تحتوي على أي معلومات حول كيف وصلنا لهذه الحالة أو كم كلفنا الوصول أو ما هو العمق في شجرة البحث.

❖ **الـ node** تمثل عقدة في شجرة البحث فهي ليست فقط حالة بل تحوي المعلومات التالية:

- 1 - state: الحالة المرتبطة بهذه العقدة.
- 2 - parent node: من أي عقدة أتينا لهذه العقدة الحالية.
- 3 - Actions: ماهي الأفعال التي نفذناها للوصول من الـ parent لهذه العقدة.
- 4 - path cost $g(x)$: تكلفة الانتقال من البداية إلى العقدة الحالية.
- 5 - Depth: عمق شجرة البحث أي عدد الخطوات من الجذر إلى النهاية.



تابع الـ Expand Function:

يقوم بأخذ كل node كدخل ويحصل من تابع Successor Fn على كل الحالات الناتجة children states يقوم بإنشاء node لكل حالة فيها:

parent	العقدة الحالية
action	الحركة المؤدية لهذه العقدة من الأب
depth	عمق الـ parent + 1
g	تكلفة الـ parent + 1

تابع الخلفاء Successor Fn:

تابع يقوم بإعطاء جميع الحالات التي يمكن الوصول إليها من حالة معينة مع الحركة التي تنتقل لكل حالة مع التكلفة يعتمد على الـ Transition model الذي تحدثنا عنه في المحاضرة السابقة لكي يظهر الحالات الناتجة عن كل حركة.

Frontier / Fringe:

في أي خوارزمية بحث (BFS, DFS, A*, UCS) لدينا مجموعة من العقد node التي:

- 1- تم توليدها بالفعل أو اكتشافها.
 - 2- التي تم اكتشافها لكن لم تتم زيارتها بعد، أي لم نقوم بعمل Expand لها هذه العقد نقوم بتخزينها بما يسمى بالـ Fringe
- إذاً الـ Fringe هو مجموعة من العقد التي تم اكتشافها لكن لم يتم زيارتها وقد يكون بشكل stack أو queue

ملاحظة (1):

- خوارزمية BFS تستخدم الـ queue كقائمة انتظار.
- خوارزمية الـ DFS تستخدم الـ stack كقائمة انتظار.
- خوارزمية UCS/A* تستخدم الـ priority queue.

ملاحظة (2):

stack: lifo → Last In First Out
queue: fifo → First In First Out



بعض العمليات التي يمكن تطبيقها على الـ queue :

1- Empty? (queue)

تتحقق إذا كانت الـ queue فارغة أم لا.

2- First (queue)

ترجع العنصر الأول في الطابور.

3- Remove-first(queue)

تعيد العنصر الأول ثم تقوم بإزالته.

4- insert (element, queue)

إضافة عنصر إلى الطابور وتعيد هذا العنصر الذي تمت إضافته

5- insert all (element, queue)

إضافة مجموعة من العناصر دفعة واحدة (مثل كل الـ children التي نتجت من expand العقدة الأب).

الهيكل العام للعمل في شجرة البحث:

1. البدء بالحالة الابتدائية:

يتم البدء من frontier تحوي الحالة الابتدائية

2. اختيار عقدة للتوسيع بناء على الاستراتيجية.

ندخل في حلقة : if(frontier=empty)

يتم إرجاع عدم وجود حل.

3. إذا كانت العقدة هدفاً ، ارجع الحل:

else

يتم إزالة العقدة من الـ frontier

(هل العقدة هي الحل) if

يتم إرجاع الهدف.

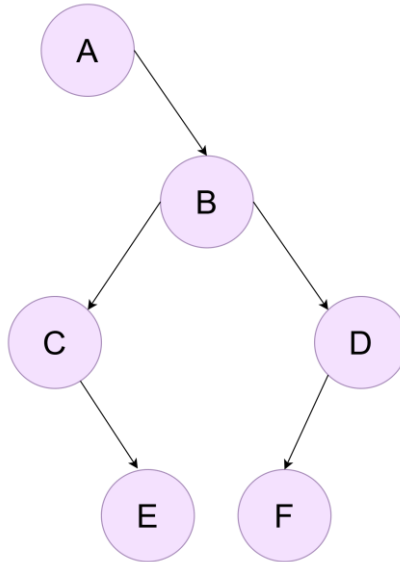
4. توسيع العقدة وإضافة العقد الجديدة إلى الشجرة.

else

وإلا يتم عمل توسعه للعقدة expand node أي إضافة العقد الناتجة عن هذه العقدة إلى frontier.

مثال للتوضيح:

نريد إيجاد الطريقة من A إلى E.



1. Frontier=A

ندخل الحلقة

- 1 ⇐ هل الـ frontier فارغ ؟ لا ، نتابع
- 2 ⇐ نزيل العقدة A من الـ frontier
- 3 ⇐ هل العقدة A هي الحل؟ لا ، نتابع
- 4 ⇐ نضيف العقدة B إلى frontier

2. Frontier=B

ندخل الحلقة

- 1 ⇐ هل الـ frontier فارغ ؟ لا ، نتابع
- 2 ⇐ نزيل العقدة B من الـ frontier
- 3 ⇐ هل العقدة B هي الحل؟ لا ، نتابع .
- 4 ⇐ نضيف العقدة C,D إلى frontier

3. Frontier=C,D

نكرر نفس الخطوات

- 2 ⇐ باعتبار الـ frontier هو queue نقوم بإزالة العقدة C أولاً.
- 3 ⇐ هل العقدة C هي الحل؟ لا ، نتابع .
- 4 ⇐ نضيف العقدة E إلى الـ frontier.

4. Frontier=D,E

نكرر الخطوات

2 ⇐ نزيل العقدة D.

3 ⇐ هل هي الحل؟ لا ، نتابع .

4 ⇐ نضيف العقدة F

5. Frontier=E,F

نكرر الخطوات

2 ⇐ نزيل العقدة E.

3 ⇐ هل هي الحل؟ نعم نعيد الحل.

الحل هو المسار A->B->C->E

العقد التي تم زيارتها هي A->B->C->D->E

Repeated State Problem:

الخوارزمية قد تدخل في حلقة لانهاية أو تكرر نفس العمل مرات عدة في حال عدم فحص تكرار الحالات.
لحل هذه المشكلة يجب تعديل منهج العمل في شجرة البحث باستخدام:

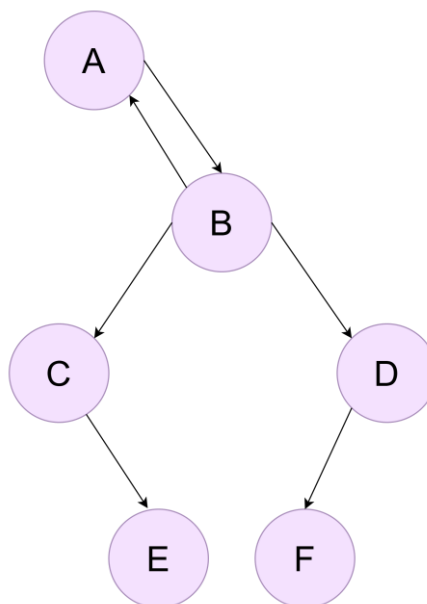
🔗 Closed List :

نضيف العقد التي تمت زيارتها بالفعل من أجل تفادي زيارتها مره أخرى.

🔗 Frontier ⇔ Open List :

حالات تم اكتشافها لكن لم يتم زيارتها بعد.

مشكلة تكرار الحالات:

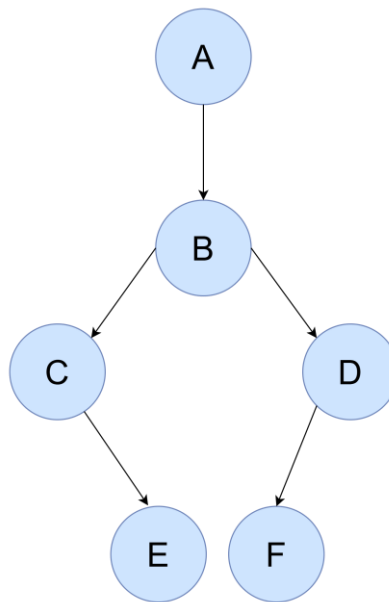
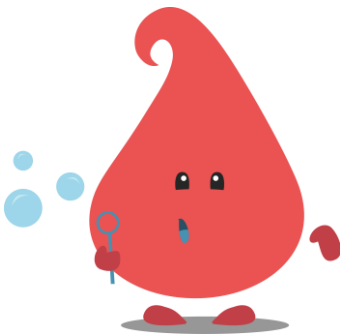


تعديل آلية العمل في شجرة البحث:

1. البدء من الـ frontier الذي يحوي حالة ابتدائية:
2. تهيئة قائمة الـ explored لوضع العقد المزارة فيها.
3. الدخول في حلقة للمرور على الحالات:
 - إذا كانت الـ frontier فارغة يتم إرجاع لا يوجد حل.
 - وإلا أخرج العقدة من الـ frontier
 - إذا كانت هذه العقدة هي الحل يتم إرجاع الحل.
 - وإلا أضف هذه العقدة إلى قائمة الـ explored وقم بعمل expand للعقدة أي أضف العقد الناتجة عنها
 - في حال لم تكن بالـ frontier بالأصل أو بالـ explored set.

مثال للتوضيح:

The frontier is stack
نريد إيجاد الطريقة من A إلى E.



1. Frontier=A ,explored set=

1 هل الـ frontier فارغ ؟ لا، نتابع

2 نخرج العقدة ونتحقق إذا كانت الهدف نرجعها وإذا لم تكن نضعها في قائمة الـ explored

3 نقوم بعمل expand للعقدة وإضافة العقد الناتجة عنها بعد التأكد من أنها ليست ضمن الـ explored set

2. Frontier=B , explored set=A

3. Frontier= (C, D) , explored set=(A,B)

4. Frontier= (C, F) , explored set=(A,B,D)

5. Frontier=C, explored set=(A,B,D,F)

6. Frontier=E, explored set=(A,B,D,F,C)

1 هل الـ frontier فارغ ؟ لا ، نتابع

2 نخرج العقدة ونتحقق هل هي الحل ؟ نعم نعيد الحل وهو في مثالنا $A \Rightarrow B \Rightarrow C \Rightarrow E$

- تسلسل الخطوات السابقة هو طريقة عمل خوارزمية الـ DFS والتي تقوم على اكتشاف زيارة أعمق عقدة في الـ frontier.

- لو كانت الـ frontier عبارة عن queue عندئذ تكون الخطوات تمثيل لخوارزمية الـ BFS والتي تقوم على زيارة العقد الظاهرة في الـ frontier أو السطحية.

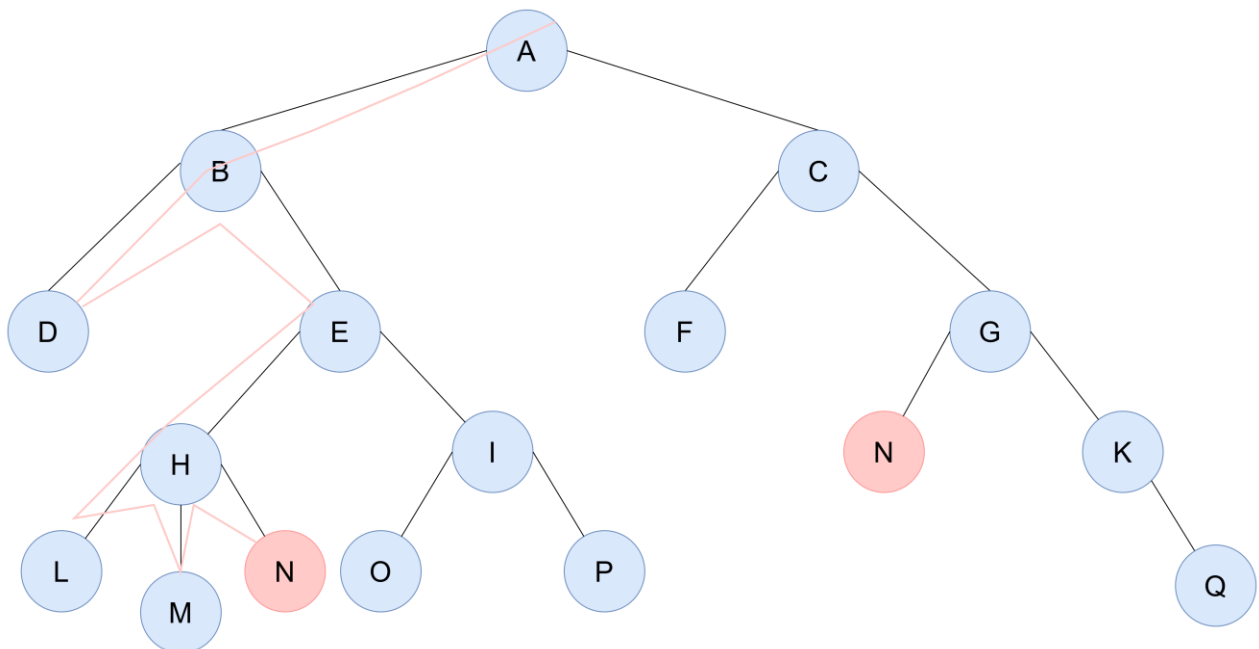
لنبدأ بالتعرف على النوع الأول من استراتيجيات البحث استراتيجيات البحث غير المطلعة: Uniformed Search Strategies لا تستخدم المعلومات المتعلقة بالمشكلة المحددة:

- Depth-first-search
- Breadth-first-search
- Uniform-cost-search

Depth-First-Search

خوارزمية البحث في العمق تقوم باستكشاف جميع الـ node في الـ Tree/Graph بطريقة تتعمق في الفروع قبل عمل backtracking .

طريقة سير هذه الخوارزمية: يسار = عمق > يمين



Properties of depth-first-search:

1- complete:

- لا، تفشل الـ DFS في العثور على حل إذا كان فضاء البحث يحتوي على أعماق لا نهائية أو في حال كان هناك حلقات بداخل المسار فإذا كان البعد لا نهائي تستمر الخوارزمية بالنزول دون العودة .
- يمكن تعديلها لتجنب تكرار الحالات (لا نقوم بزيارة عقدة كانت قد زارتها بالفعل)
- بعد التعديل تصبح الـ DFS مكتملة في فضاء الحالات المحدودة Finite State Space

2- time complexty:

- $O(b^m)$ حيث b هو عامل التفرع (متوسط عدد الحالات الناتجة عن كل حالة أي نقصد الخلفاء) و m هو أقصى عمق للشجرة
- التعقيد الزمني للخوارزمية يعتبر سيء إذا كان m (أقصى عمق) أكبر بكثير من d (عمق الحل) لأنها قد تبحث في مسارات طويلة جداً قبل أن تعثر على الحل الأقصر.

3- space complexty:

$O(bm)$ التعقيد هنا خطي linear space وهذا ما يميز الخوارزمية.

- خطي لأن الـ DFS تستخدم الـ stack في عملية التخزين

4- optimal:

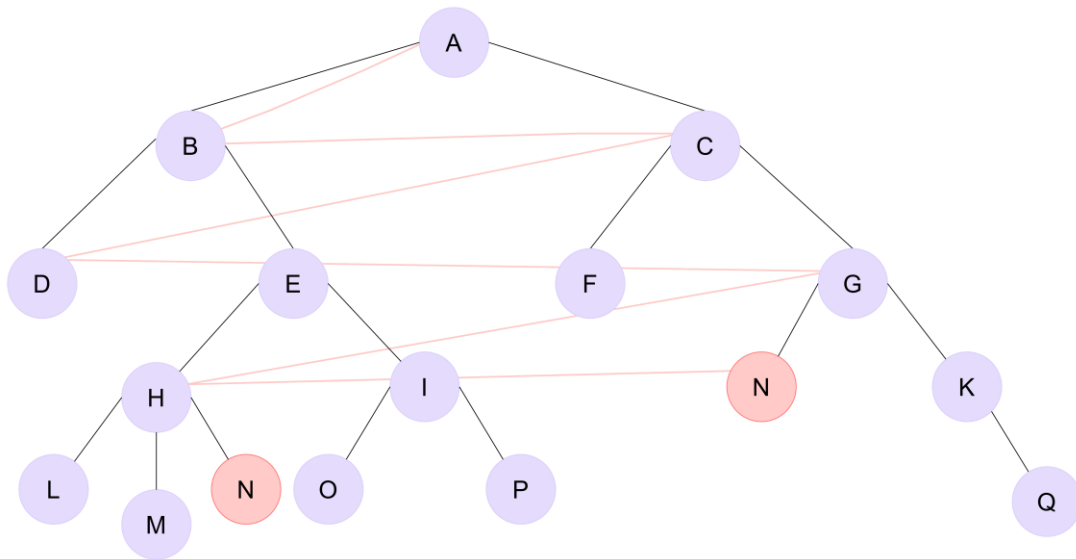
لا، خوارزمية البحث قد تجد حلاً صحيحاً لكنه ليس أفضل حل. فهي تكتفي بأول حل تجده.



تأتيك الأشياء على قدر سعيك لا على قدر رغبتك

Breadth-First-Search

خوارزمية البحث في العرض تستخدم لاستكشاف العقد في Graph أو Tree على شكل طبقات Levels.



Properties of breadth-first-search:

1- complete:

مضمونة في إيجاد الحل لأنها تفحص كل العقد في مستوى قبل الانتقال إلى المستوى الأعظم التالي .
وبالتالي لا تقع في حلقة لا نهائية.
يشترط لتكون الخوارزمية مكتملة أن يكون عدد الحركات الممكنة محدود أي عامل التفرع b is finite

2- time complexity:

$O(b^{d+1})$ سيء تستغرق وقت طويل لأنها تفحص كل العقد حتى عمق الحل d .

3- space complexity:

$O(b^{d+1})$ هو أكبر عيب لخوارزمية BFS حيث تقوم بالاحتفاظ بكل العقد في المستوى الحالي قبل أن تتمكن من المتابعة

4- optimal:

نعم لأنها تقوم بإيجاد أقصر مسار بأقل عدد من الخطوات.
يشترط أن تكون كلفة جميع الانتقالات لأي مستوى متساوية وأن تزداد كلفة الحل بزيادة عمق الوصول إليه

Uniform-Cost-Search

خوارزمية البحث بالتكلفة الموحدة وهي خوارزمية بحث عامة تهدف إلى إيجاد المسار الأقل تكلفة من عقدة البداية إلى عقدة الهدف.

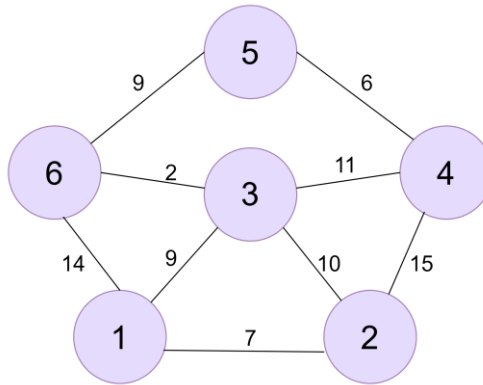
- قائمة الانتظار هنا هي Queue مرتبة بحسب تكلفة المسار وتسمى بـ priority queue
- $g(\text{node})$ تشير إلى التكلفة الاجمالية للوصول إلى هذه العقدة من البداية (الجذر).

ملاحظة:

تكون مكافئة لخوارزمية BFS في حال كانت كلفة كل الخطوات متساوية.

مثال:

لدينا الـ Graph التالي ونريد الانتقال من العقدة 1 إلى العقدة 5 باستخدام خوارزمية البحث بالتكلفة الواحدة



ذكرنا سابقاً أن البحث بالتكلفة الواحدة هو اختيار العقد ذات التكلفة الأقل ولا ننظر إلى العمق الذي تقف عنده العقدة وإنما تكلفة الوصول إليها.

- نبدأ من العقدة 1 :
هل هي الهدف؟ لا نتابع .
نلاحظ أنه يوجد ثلاث فروع تنطلق من العقدة 1 . نختار الفرع ذو التكلفة الأقل وهو الفرع ذو التكلفة 7.
- العقدة 2:
هل هي الهدف ؟ لا نتابع
نختار الفرع ذو التكلفة الأقل وهو ذو التكلفة 10 و $g(2)=7$

ملاحظة:

الفرع الذي يؤدي إلى عقدة سبق وأن زرناها لا نأخذه

■ العقدة 3:

هل هي الهدف ؟ لا نتابع

لكن نلاحظ هنا أن $g(3)=17$ ويمكن أن تكون $g(3)=9$ إذا سلطنا المسار ذو التكلفة 9 ومن العقدة 1 لذلك نعود للعقدة 1 ونختار المسار ذو التكلفة 9 .

هل العقدة 3 هي الهدف ؟ لا نتابع

نأخذ المسار ذو التكلفة 2 و $g(3)=9$

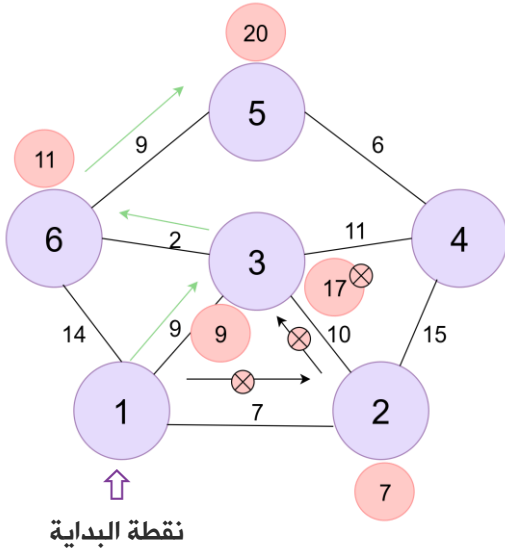
■ العقدة 6:

هل هي الهدف ؟ لا نتابع.

نأخذ المسار ذو التكلفة 9 و $g(6)=11$

■ العقدة 5:

هل هي الهدف ؟ نعم تم الوصول للهدف بكلفة $g(5)=20$



Properties of Uniform-Unit-Search:

1- complete:

أجل طالما كانت جميع التكاليف $\epsilon \leq cost$ أي لا يوجد خطوة بتكلفة صفرية أو سالبة لأن الخوارزمية ستدخل في حلقة لا نهائية في حال عدم استخدام آلية تمييز للـ visited node.

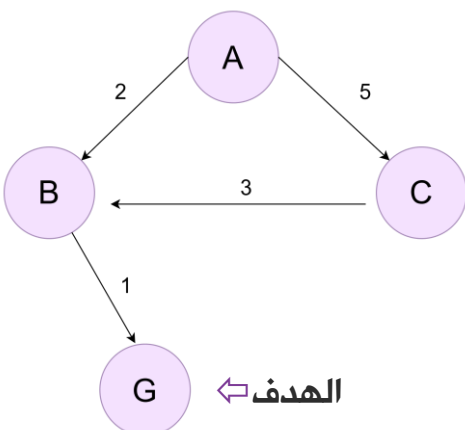
2- time complexity:

يقاس بعدد العقد التي يتم توسيعها قبل الوصول إلى العقدة الهدف

■ الخوارزمية توسع العقد بترتيب تصاعدي حسب كلفة المسار $g(n)$ وبالتالي فإن أي عقدة كلفتها $g(goal) \geq g(n)$ أي كلفتها أصغر من كلفة الحل الأمثل سوف يتم توسيعها أثناء البحث.

■ مثال:

هنا $g(G)=3$ سيتم توسيع العقد A و B لأن $g(A)=0 < 3$, $g(B)=2 < 3$.
بينما C لن يتم عمل توسيع لها لأن $g(C)=5 > 3$.



3- space complexity:

تشغل بعدد العقد التي يتم توسعتها قبل الوصول إلى العقدة الهدف.

4- optimal:

نعم , لأنها توسع العقد المرتبة بشكل تصاعدي حسب الـ $g(n)$ لذلك أول هدف تصل إليه هو الأقل كلفه دوماً.

Blind Search Algorithm

```

1 OPEN = {InitialState}
2 CLOSE = nil
3 SUCCESS = False
4 While OPEN != {} And SUCCESS=False do
5 Begin
6   u = Get_First_Node_From_OPEN(OPEN)
7   OPEN = Queue(OPEN)
8   if GoalState (u) Then
9     SUCCESS=True
10  ELSE
11    Begin
12      Add(u, CLOSE)
13      For each v in Children(u) do
14        Begin
15          If v Not in CLOSE Then
16            Begin
17              Add (v, OPEN)
18              g(v) = g(u) + k(u,v)
19              Father(v)=u
20            End
21          End
22        End
23      End
24    If OPEN={} Then FAIL
25  Else
26    GenerateSolutionPath(u)
    
```

هذا الشكل من الكود يسمى بـ pseudocode وهو طريقة كتابة الخوارزميات بلغة مفهومة للإنسان بدون التزام بقواعد لغة برمجة معينة.

السطر 1: إنشاء قائمة باسم open ووضع الحالة الابتدائية فيها.

السطر 2: إنشاء قائمة باسم فارغة close .

السطر 3: في حال وجدنا الحالة الهدف تصبح True

السطر 4: طالما أن open ليست فارغة ولم نصل إلى الهدف نفذ ما بداخل الحلقة

السطر 6: اختيار عقدة عن القائمة open تبعاً لسياسة البحث المتبعة.

السطر 7: تحديث القائمة open بعد إزالة u منها.

السطر 8-9: فحص العقدة u في حال كانت الهدف.

السطر 10-11-12: إذا لم تكن u الهدف أضف العقدة إلى القائمة close.

السطر 13: المرور بحلقة for على أبناء u (العقد التي نستطيع الوصول إليها من u).

السطر 15-16-17: إذا لم يكن الأبن ضمن قائمة العقد التي تمت زيارتها أضفه للقائمة open

السطر 18: تكلفة الوصول لـ v

السطر 19: تعيين والد v في الشجرة هو u

السطر 24: إذا انتهت عناصر open ولم تصبح

الـ success تساوي True الناتج هو الفشل أي لا يوجد حل ضمن فضاء البحث.

السطر 25-26: إذا وجد حل نعيد بناء المسار أي في حال

success = True نستدعي إجراء لإعادة بناء مسار الحل

كيف؟ تتبع الـ father(u) للخلف حتى نصل إلى Initial State ثم نقلب المسار ليصبح من البداية إلى الهدف.

$$g(v) = g(u) + k(u, v)$$

- $g(u)$ تكلفة الوصول إلى u من البداية.
- $k(u, v)$ هي تكلفة الانتقال من u إلى v .
- $g(v)$ التكلفة من البداية حتى v .

ملاحظة:

Begins تشير إلى بداية بلوك أو الكتلة و End تشير إلى نهاية الكتلة مثلاً:

```
1 if x>5 Then
2   Begin
3     print('Big')
4   End
```

- ذكرنا سابقاً أن تابع Get_First_Node_From_OPEN يأتي بعقدة من القائمة open بناءً على استراتيجية البحث المتبعة مثلاً:
- 1 DFS يعيد آخر عنصر أضيف للقائمة (LIFO)
- 2 BFA يعيد أول عنصر أضيف للقائمة (FIFO)
- 3 UCS يتم ترتيب العناصر بـ OPEN بحسب تكلفة $g(v)$ ثم يتم اختيار العقدة ذات التكلفة الأقل.



**فكل ناءٍ قريب إن صبرت له
وكل صعب إذا قاومته هانا**

<See You Soon>